Algorithm Design & Analysis

classmate
Date
Page

⊙ Introduction:-

⊙ Algorithms:-

⊙ An algorithm is a well-defined, finite sequence of instructions designed to perform a specific task or solve a particular problem.

→ Manipulation of Inputs:-

* Definiteness. (Input ⇒ Instance)
* Finiteness.
* priori Analysis of Algorithm.
* posteriori Analysis of Algorithm.

⊙ In Mathematics and computer science, an algorithm is a finite sequence of well-defined, computer-implementable instructions, typically to solve a class of problem or to perform a computation

→ Will accept zero or more input, but generate at least one output

Ex:- generate random number.

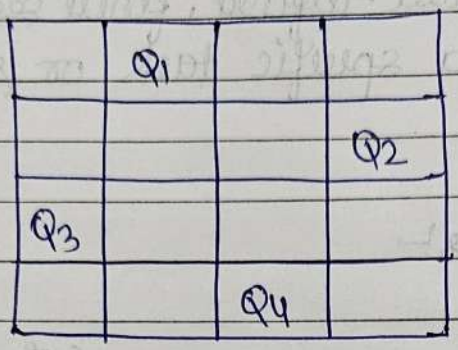
⊙ properties of Algorithms:-

1. Should terminate in finite time.
2. Unambiguous.
3. Input zero or more output at least one.
4. Every instruction in algorithm should be effective.
5. It should be deterministic

Ques. Four Queens Q_1, Q_2, Q_3, Q_4

constraints :- 1. No 2 queens in one row.
2. No 2 queens in one diagonal.
3. No 2 queens in one columns.

Ans.

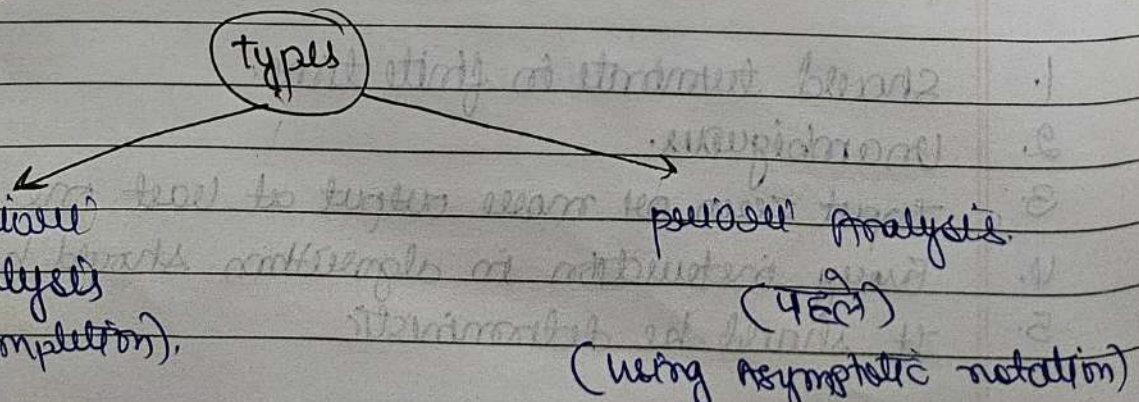


⊙ Analysis of an Algorithm :-

⊙ we do analysis of algorithm to do a performance comparison between different algorithm to figure out which one is best possible option.

→ Following parameter :-

1. Time.
2. Space.
3. Bandwidth.
4. Register.
5. Battery power.



Asymptotic Analysis:-

- In this we evaluate the performance of an algorithm in terms of input size (we don't measure the actual time) we calculate, how does the time (or space) taken by an algorithm increase with the input size.
- ∴ pen and paper mode

Asymptotic Notation:-

- Behaviour of function when the values are approches to '∞'.

- are mathematical tools to represent the time complexity of algorithms for Asymptotic analysis.

1. Big oh (O).
2. Big omega (Ω).
3. Theta (θ).
4. Little oh (o).
5. Little omega (ω).

1. Big oh(O):-

- f(n) & g(n) are two functions
  f(n) is O(g(n)) iff.

$$f(n) \leq c \cdot g(n) \quad \forall n \geq K \text{ \&\& } \exists c$$

2. Theta Notation θ.
3. Omega Notation Ω.

classmate

Date

Page

→ Recurrence Relations

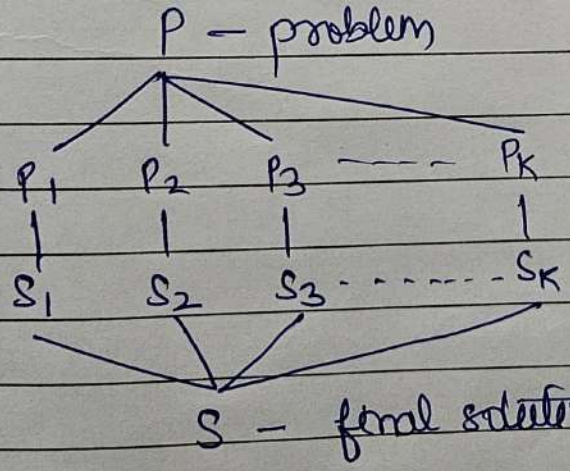
1. Recurrence tree
2. Master method.
3. Iterative method.

Divide and conquer :-

It is a fundamental algorithm design paradigm where a problem is broken into smaller subproblems, solve independently, and then these result are combine to form the solution to the original problem.

General structure (pseudocode) :-

```
Divide and Conquer (Problem)
    if problem is small enough:
        solve it directly
    else:
        divide problem into subproblems
        solve each subproblem recursively
        combine the solutions.
```



– subproblem. (may or may not be equal)

$T(n) = D + T(n_1) + T(n_2) + C$.

$T(n) = T(n_1) + T(n_2) + f(n)$.

where ; D = Divide time.
C = Conquer time.

Ex:- Algorithms:-

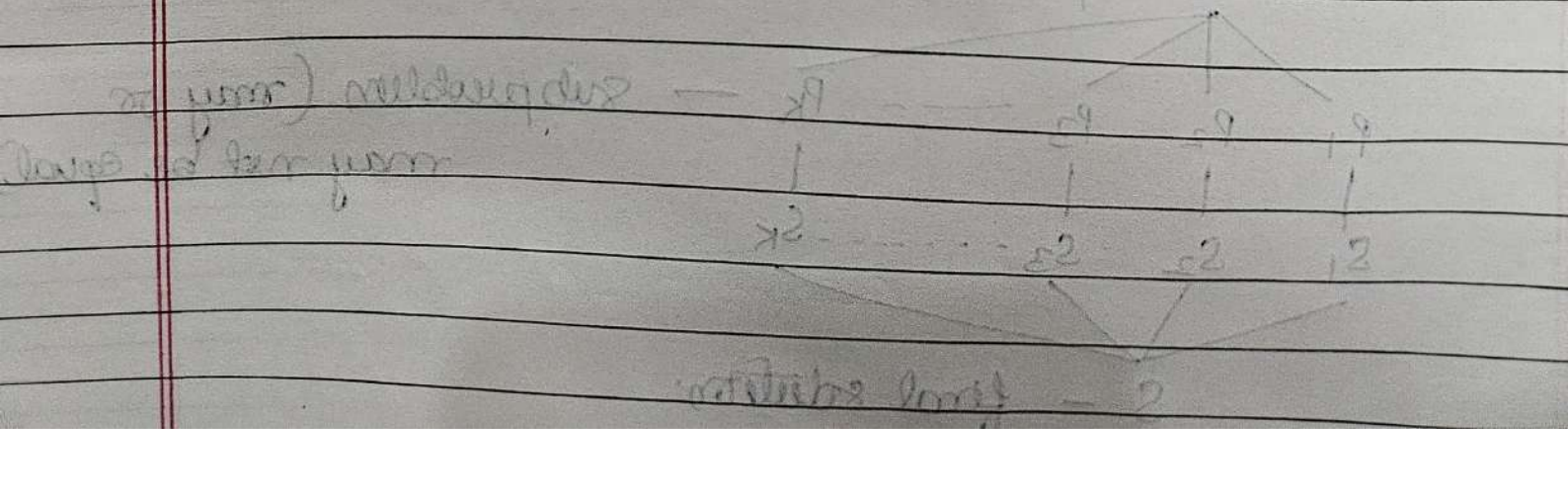
1. Quick sort.
2. Merge sort.
3. Binary sort.
4. Strassen's Matrix Multiplications.
5. Karatsuba Multiplication.

⊙ Advantages

1. Efficient for large problems.
2. Easy to parallelize.
3. Reduce time complexity compared to brute force.

⊙ Disadvantages

1. Recursive calls use more memory (stack).
2. overhead of subproblems are not balanced.



Quick Sort

⊙ Sorting and order Statistics:-

1. Quick Sort:-

⊙ It is a Divide and conquer based recursive sorting algorithm.

→ In this algorithm we

1. choose a pivot element.
2. partition the array into two parts
   - left – smaller.
   - Right – greater.
3. Recursively apply quick sort to the left and Right until get sorted array.

Divide → pick a pivot, split the array into two subarray.
conquer → Recursively sort both subarrays.
combine → Merge the sorted part.

Ex:- [8, 3, 1, 7, 0, 10, 2]

pivot 7
left - [3, 1, 0, 2]
Right - [8, 10]

sorted

pivot 3
left → [1, 0, 2]
Right → [7]

pivot 1
left [0, 1, 2]
Right [3, 7]

Then merge

[0, 1, 2, 3, 7, 8, 10]

⊙ Time complexity:-

1. Best → $O(n \log n)$
2. Average → $O(n \log n)$
3. Worst → $O(n^2)$

→ Pseudo-code

```
quicksort(arr, start, end):
    If start >= end
        return
    pivotIndex = partition(arr, start, end)
    quicksort(arr, start, pivotIndex - 1)
    quicksort(arr, pivotIndex + 1, end)
```

quicksort

```
function partition(arr, low, high)
    pivot = arr[high]
    i = low - 1
    for j = low to high - 1:
        If arr[j] <= pivot:
            i++
            swap arr[i], arr[j]
    swap arr[i+1], arr[high]
    return i+1
```

partitioning

⇒ $T(n) = T(k) + T(n-k-1) + O(n)$

⇒ where

⇒ k = size of the left subarray.

⇒ $n-k-1$ = size of right subarray.

Note:-
1. Inplace Algorithms
2. Astable.

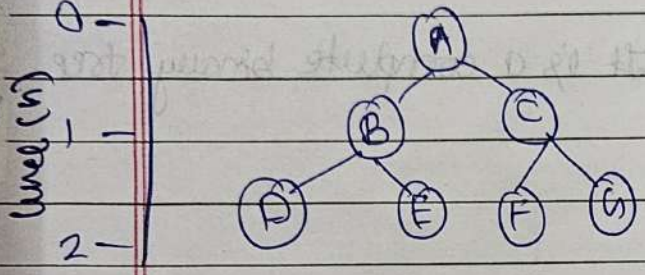
HEAP SORT

classmate
Date
Page

⊙ Heap sort:-

⊙ Heap sort is a comparison-based sorting algorithm that uses a binary heap data structure.

| A | B | C | D | E | F | G |



If a node is at index (i)
its left child is at $(2*i)$
its right child is at $(2*i+1)$
its parent is at $\left\lceil \frac{i}{2} \right\rceil$.

∴ Full Binary tree

no. of nodes in Full Binary tree = $2^{h+1}-1$
where, h = no. of levels.

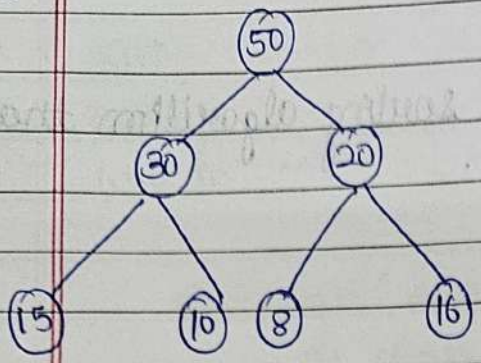
Note:- Every Full Binary tree is a complete Binary tree.

→ Complete binary tree:-
↳ There should be no missing value in the array element.
height = $\log n$

⊙ Heap:-

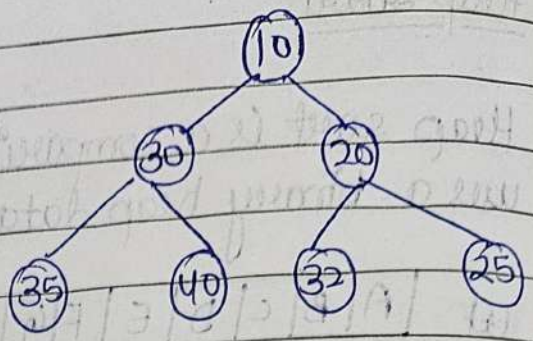
1. Max Heap.
2. Min Heap

Max Heap



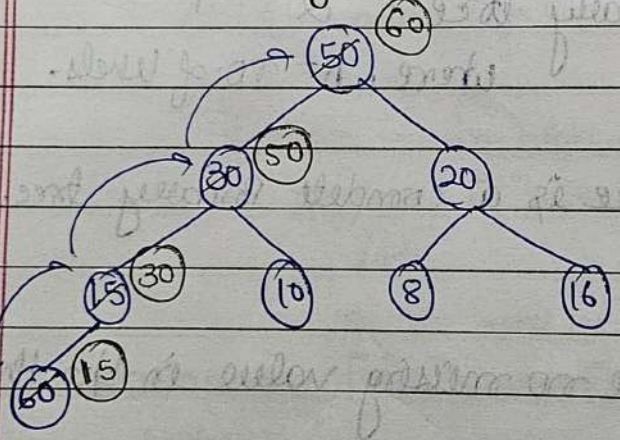
→ It is a complete binary tree.

Min Heap



→ It is a complete binary tree.

→ Insert in any heap



If we want to add 60 in the heap.
we add it at the last

→ time taken to insert an element in an any heap.

O(1) to O(log n)

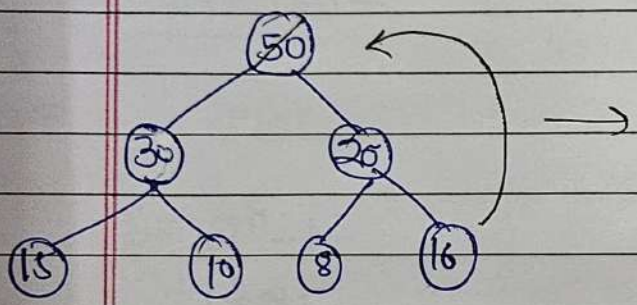
Note: direction of adjustment is always upwards.

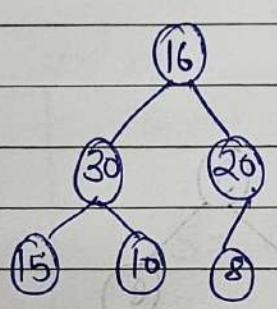
Q) Delete in heap (element)

→ We always delete a root element.
then, last element take its place
Then we adjust it as requirement (max/min).

Note:- Direction of adjustment is always downwards.

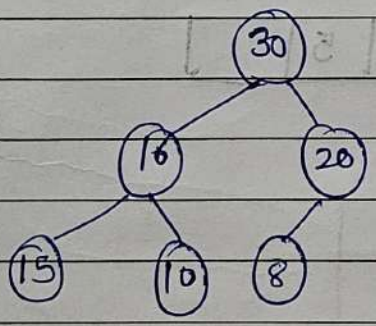
time (logn)





50	30	20	15	10	8	16

16	30	20	15	10	8	50



Final

30	16	20	15	10	8	50

→ Heap sort

Step 1:- Build a Max-Heap from the given array
↳ In a max-heap, the largest element is always at the root.

Step 2:- Repeatedly extract the maximum

1. Swap the root element with the last element.
2. Reduce the heap size by 1.
3. Heapify the root again to maintain the max-heap property.

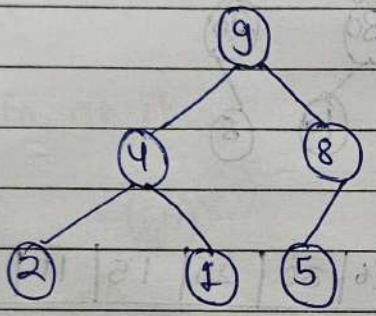
→ We can also say that first create the max heap then delete the element one-by-one, you get the sorted array.

Ex:-

1	2	3	4	5	6
1	9	8	2	4	5

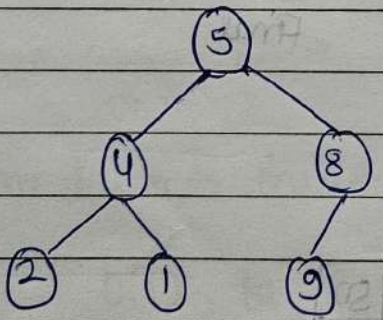
→ Max heap

Step1.



9	4	8	2	1	5

Step2



5	4	8	2	1	9

→ sorted.

→ Repeat the process, to get sorted array.

Date
Page

⊙ **Heapify:**

⊙ Heapify is a process of arranging the elements of a binary tree / array in such a way that it satisfies the heap property.

$T(n) = O(n)$

→ Max & Min size of array to store binary tree & CBT, FBT.

1. **Maximum:**

*	CBT	FBT
	$S = 2^h - 1$ where, h = height of tree	$S = 2^h - 1$ where, h = height of tree.

2. **Minimum:**

*	CBT	FBT
	$S = 2^{(h-1)}$ where h = height of tree	$S = 2^h - 1$ where, h = height of tree.

⊙ **Time complexity:**

⊙ Insertion one Node in Empty heap → $O(1)$

⊙ Insertion one Node in Maxheap / Minheap → $O(\log n)$

- Inserting n Node in max heap / Min heap → $O(n \log n)$
- Deleting root Node → $O(1)$.
- Deleting Root Node of max heap / Min heap = $O(\log n)$
- Deleting n Node → $O(n \log n)$

→ Why T.C Heapify is $O(n)$:-

$$\Rightarrow \sum_{0}^{\log n} \left[\frac{n_{total}}{2^{h+1}} \right] \times O(h)$$

$$\Rightarrow \sum_{0}^{\log n} \left[\frac{n_{total}}{2^h \cdot 2} \right] \times ch$$

$$\Rightarrow \frac{nc}{2} \sum_{0}^{\log n} \left[\frac{h}{2^h} \right]$$

$$\Rightarrow \frac{nc}{2} \sum_{0}^{\infty} \left[\frac{h}{2^h} \right]$$

$$\Rightarrow \frac{nc}{2} \times 2$$

$$\Rightarrow \boxed{O(n)}$$

- Priority queue

→ smaller number → Higher

larger number → Higher

Min heap

max heap.

→ In Normal queue → FIFO.

→ In a priority queue → the element with the highest priority is served first

→ Pseudocode for Heap sort:

```
HEAPSORT (A){
    Build-Max-heap (A)
    for i = n down to 2 {
        Exchange A[1] with A[i]
        A.heapsize = A.heapsize - 1
        Max Heapify (A, 1).
}
```

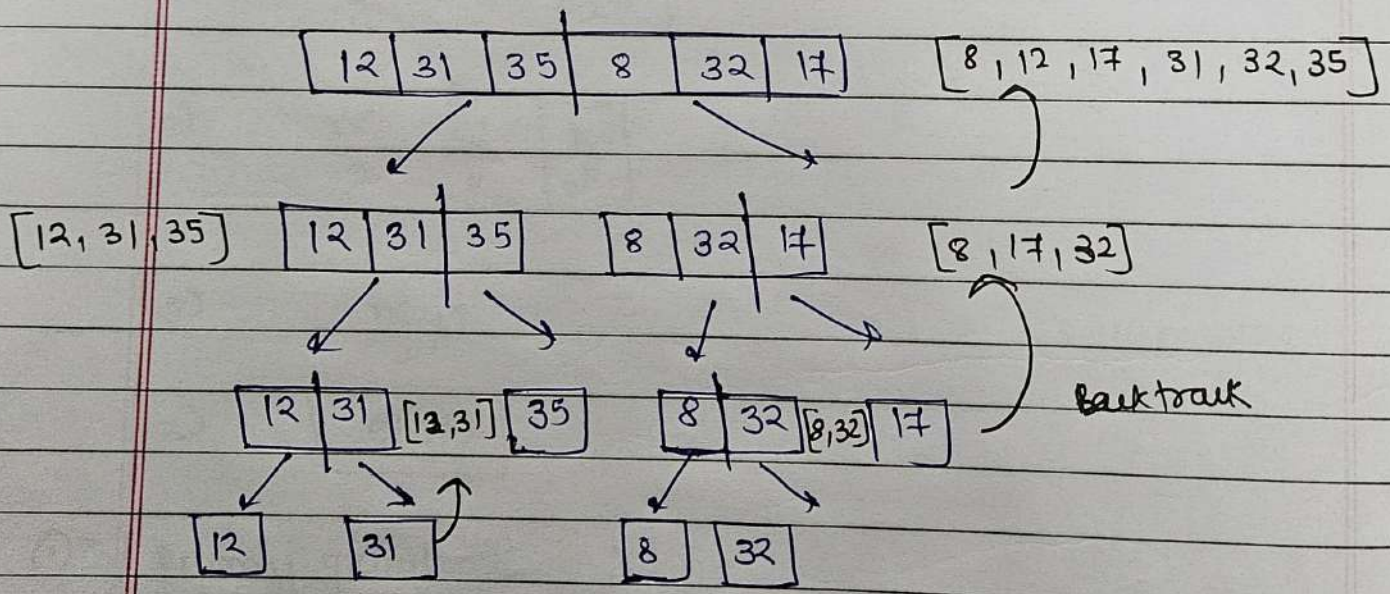
Merge Sort

- Merge Sort:
- It divides the array into smaller subarrays, until it get single element per array, and then sort it and merge together to get the sorted array.
- Out of place Algorithm.
- Stable Algorithm.

→ Divide and conquer based Algorithm.

Ex:- | 12 | 31 | 35 | 8 | 32 | 17 |

→ arr[] = {12, 31, 35, 8, 32, 17}



→ Steps:

1. Divide until get single element array.
2. Do backtracking to get final sorted array.

⊙ Pseudocode :-

→ Merge sort :-

```
void mergeSort ( arr[], start, end){ . if (st<end){
        int mid = st + (end-st)/2

        mergeSort (arr[], st, mid)
        mergeSort (arr[], mid+1, end)
        merge (arr, st, mid, end)
    }
}
```

→ Merge step :-

```
void merge (arr, st, mid, end){
    vector int> temp ;
    i = st, j = mid+1;
    while (i<=mid && j<= end){
        if (A[i]<= A[j]){
            temp . pb (A[i])
            i++;
        } else {
            temp.pb (A[j])
            j++
        }
    }
while (i<=mid){
    temp. pb (A[i])
    i++;
}
while (j<=end){
    temp.pb (A[j])
```

Final :-

```
for (idx=0; idx<temp.size(); idx++)
    A [idx+st] = temp [idx];
```

⊙ Time complexity:-

1. Best case :- $O(n \log n)$.
2. Average case :- $O(n \log n)$.
3. Worst case :- $O(n \log n)$.

Counting Sort

classmate
Date
Page

⊙ Count sort:-

⊙ Counting sort is a non-comparison sorting algorithm.
→ one of the fastest algorithm.

* The range of numbers (k) is not much larger than the number of element (n) then it will work well.

⊙ Main idea:-

1. Count how many times each number appears.
2. Store this count in an auxiliary array.
3. Use these counts to place numbers in the correct sorted array.

Ex:-

0	1	2	3	4	5	6
3	1	9	7	1	2	4

→ max=9.

→ then make a count array. (initialize all the value to zero then)

0	1	2	3	4	5	6	7	8	9
0	2	1	1	1	0	0	1	0	1

→ then create a array for final sort.

0	1	2	3	4	5	6
1	1	2	3	4	7	9

→ Not - In-place algorithm.
→ Stable sorting algorithm.

* Pseudocode:-

⊙ countsort() {
for (i=0; i<n; i++) {
++ count [a[i]];
}
for (i=1; i<=k; i++) {
count [i] = count [i] + count [i-1]
}.

Type-1.

OR.

⊙ countsort (A, k)
let count [0..k] = {0}.
for i = 1 to length(A):
count [A[i]] += 1
for i = 1 to k:
count [i] = count [i] + count [i-1]

Type-2.

→ then,
for j = length (A) down to 1;
output [count [A[j]]] = A[j]
count [A[j]] -= 1
return output.

For both type

* Time complexity:-

→ O(n+m)
where,
m = max value
n = size of input array.

* Space compl:-

→ O(n+m) → extra space used.

Shell Sort

classmate
Date
Page

⊙ **Shell sort:-**

→ Based on Insertion sort algorithm

→ shell sort is an in-place comparison-based sorting algorithm.

→ Gap = floor $(n/2)$

where

n = no. of element in array.

Ex:-

0	1	2	3	4	5	6	7	8
15	19	20	38	24	41	30	31	12

$n = 9$

→ gap in pass 1 $= \left[\frac{n}{2}\right] = \left[\frac{9}{2}\right] = 4$

a[0] ⟶ a[4]
a[1] ⟶ a[5]
a[2] ⟶ a[6]
a[3] ⟶ a[7]
a[4] ⟶ a[8]

} compare

12	19	20	31	15	41	30	38	24
0	1	2	3	4	5	6	7	8

(∴ after one pass)

→ gap in pass 2 $= \frac{\text{pass 1 gap}}{2} = \frac{4}{2} = 2$

a[0] ⟶ a[2]
a[1] ⟶ a[3]
a[2] ⟶ a[4]
a[3] ⟶ a[5]
a[4] ⟶ a[6]
a[5] ⟶ a[7]
a[6] ⟶ a[8]

} Compare

12	19	15	31	20	38	24	41	30	(∵ after pass2)
0	1	2	3	4	5	6	7	1	

→ gap in pass 3 = $\frac{\text{pass 2 gap}}{2} = \frac{2}{2} = 1$.

a[0] → a[1]
a[1] → a[2]
a[2] → a[3]
a[3] → a[4]
a[4] → a[5]
a[5] → a[6]
a[6] → a[7]
a[7] → a[8]

compare.

12	15	19	20	31	24	38	30	41	(∵ after pass3)

12	15	19	20	24	31	30	38	41

12	15	19	20	24	30	31	38	41	→ final

① pseudocode:

```
shell sort (A, n)
    for (gap = ⌊n/2⌋; gap > 0; gap = gap/2) {
        for (i = gap, i < n, i++) {
            temp = A[i]
            for (j = i; j >= gap && A[j-gap] > temp; j = j-gap)
            {
                A[j] = A[j-gap]
            }
            A[j] = temp;
        }
    }
```

⊙ Time complexity :-

1. Best :- $O(n \log n)$.
2. Average :- $O((n)^{3/2})$.
3. Worst :- $O(n^2)$.

⊙ Space complexity :-

⊙ $O(1)$.

Radix Sort

Radix Sort:-

Radix sort is a non-comparison-based sorting algorithm. It sorts numbers digit by digit:-

→ starting from the least significant digit to most significant digit. (vice-versa in some variants)

→

Ex:-

15	1	321	10	802	2	123	90	109	11

→ Find out the maximum number. then, find how many digits are there in that no.

→ Then make all the number equal to the max number in digits with the help of '0'.

015	001	321	010	802	002	123	090	109	011

no. of pass = no. of digit in maximum number.

→ Pass 1:- Fill Bucket according to first no./digit.

0 : 010, 090	
1 : 001, 321, 011	Then,
2 : 802, 002	
3 : 123	010 | 090 | 001 | 321 | 011 | 802 | 002 | 123 | 015 | 109
4 :	
5 : 015	
6 :	
7 :	
8 :	
9 : 109	

→ pass 2:- Fill bucket according to 2nd digit

0: 001, 802, 002, 109
1: 010, 011, 015
2: 321, 123
3:
4:
5:
6:
7:
8:
9: 090

001	802	002	109	010	011	015	321	123	090

→ pass 3:- Fill bucket according to 3rd digit.

0: 001, 002, 010, 011, 015, 090
1: 109, 123
2:
3: 321
4:
5:
6:
7:
8: 802
9:

1	2	10	11	15	90	109	123	321	802

→ It use countsort as a subroutine.

⊙ pseudocode:-

```
RadixSort (arr, size) {
    1. take arr[size]
    2. Get max from the array.
        m = GetMax [arr, size]
    3. Do counting sort for every digit
        for (int div = 1; m/div > 0; div *= 10) {
            countingsort (arr, size, div)
        }
}
GetMax (arr, size) {
    1. Max = arr[0]
    2. for (int i = 1; i < size; i++) {
        if (arr[i] > max) {
            max = arr[i]
        }
    }
    3. return max.
}
```

⊙ Time complexity:-

→ $O(d \times (n+k))$

where, d = no. of digit in the largest number
n = number of elements
k = range of digit value. (10 for decimal)

Note →
→ Stable. Algorithm.
→ Not in place algorithm.

Bucket Sort

⊙ Bucket sort :-

⊙ Bucket sort is a sorting algorithm that distributes elements into several groups called buckets. Each bucket is then sorted individually, and finally, all buckets are concatenated to get the sorted result.

Eg - | .11 | .9 | .21 | .8 | .17 | .19 | .13 | .1 | .24 | .12 | .21 | .22 |

→ works on floating number between range 0.0 to 1.0

① Ex- | 0.79 | 0.13 | 0.64 | 0.39 | 0.20 | 0.89 | 0.53 | 0.42 | 0.06 | 0.94 |

Buckets ↑

0 → 0.06
1 → 0.13
2 → 0.20
3 → 0.39
4 → 0.42
5 → 0.53
6 → 0.64
7 → 0.79
8 → 0.89
9 → 0.94

→ Sorted array :-

| 0.06 | 0.13 | 0.20 | 0.39 | 0.42 | 0.53 | 0.64 | 0.79 | 0.89 | 0.94 |

Not suitable if data is skewed.

① pseudocode :-

1. Let B[0.... n-1] be a new array
2. n = A.length
3. for i = 0 to n-1
   make B[i] an empty list
4. for i = 1 to n
5. Insert A[i] into list B[⌊n A[i]⌋]
5. for i = 0 to n-1
6. Sort list B[i] with insertion sort
7. Concatenate the list together.
   B[0], B[1] .... B[n-1] (in order)

② Ex:-

0.42	0.32	0.23	0.52	0.25	0.47	0.51	0.01	0.85	0.91

0 → 0.01
1 →
2 → 0.23, 0.25
3 → 0.32
4 → 0.42, 0.47
5 → 0.52, 0.51
6 →
7 →
8 → 0.85
9 → 0.91

Sort the array element in each Bucket.

0 → 0.01
1 →
2 → 0.23, 0.25
3 → 0.32
4 → 0.42, 0.47

5 → 0.51, 0.52
6 →
7 →
8 → 0.85
9 → 0.91

⇒ Final sorted array:

0.01	0.23	0.25	0.32	0.42	0.47	0.51	0.52	0.85	0.91

⊙ Time complexity :-

1. Best → $O(n+k)$.
2. Average → $O(n+k)$.

where, n = number of elements.
k = number of buckets.

3. Worst → $O(n^2)$.

⊙ Space complexity :-

→ $O(n+k)$
↳ no. of buckets.

Advanced Data Structures :-

Red-Black Tree

classmate
Date
Page

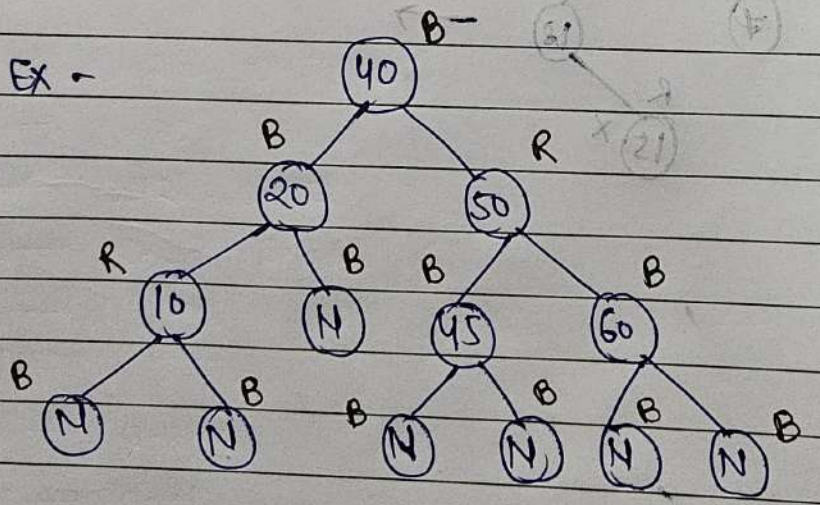
Red-black tree :-

Red-black tree is a type of self-balancing Binary Search Tree (BST). It keeps the tree balanced using colour rules (red + black nodes)
This ensures operations like search, insert, delete happen in O(log n) time.

Rules :-

1. Every node is either Red or Black.
2. The root is (root) always black.
3. Every leaf (NIL) is black. (NIL-black)
4. If a node is red, then both its children are black.
5. Every path from root → leaf must have the same numbers of black nodes

Ex -



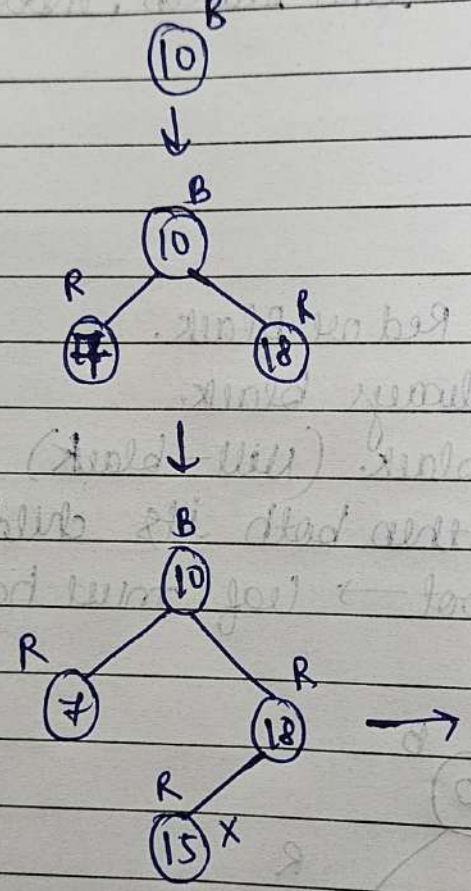
∴ B- Black.
∴ R- Red.

→ To understand anything further in Red Black tree we need to know about Binary tree and AVL tree.

Insert in R-B-tree

10, 20, 30, 15, 25, 5, 12, 35, 40, 32, 50, 11.

Ex 2 10, 18, 7, 15, 16, 30, 25, 40, 60, 2, 1, 70.



B Tree

classmate
Date
Page

⊙ B Tree:-

⊙ A B-tree is a self-balancing search tree in which nodes can have multiple keys and children

→ It is widely used in databases and file systems because it reduces the number of disk accesses.

Binomial Heap

⊙ Binomial Heap:-

⊙ Binomial Heap is a collection of Binomial Tree, called Recursively.

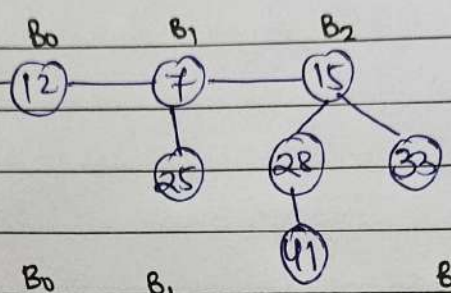
→ Satisfies "min-Heap property".

→ Lets understand what is Binomial Tree:-

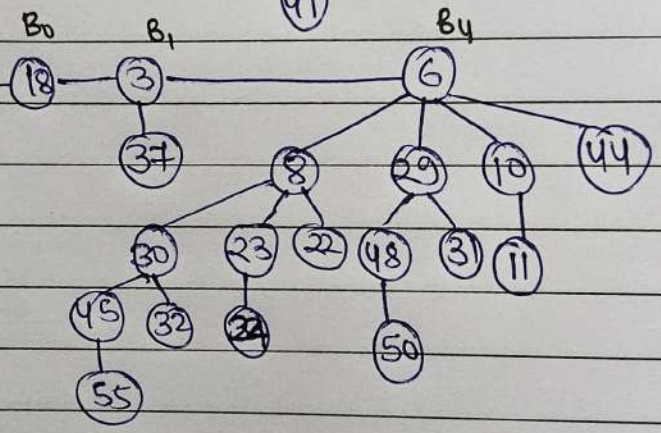
⊙ operations :-

1. Creating a New Node.
2. Finding a Minimum Key.
3. Union of two Binomial Heap.
4. Inserting a Node in B-Heap.
5. Extracting Minimum Key.
6. Decreasing a key.
7. Deleting a Node.

=> Heap (H_1)

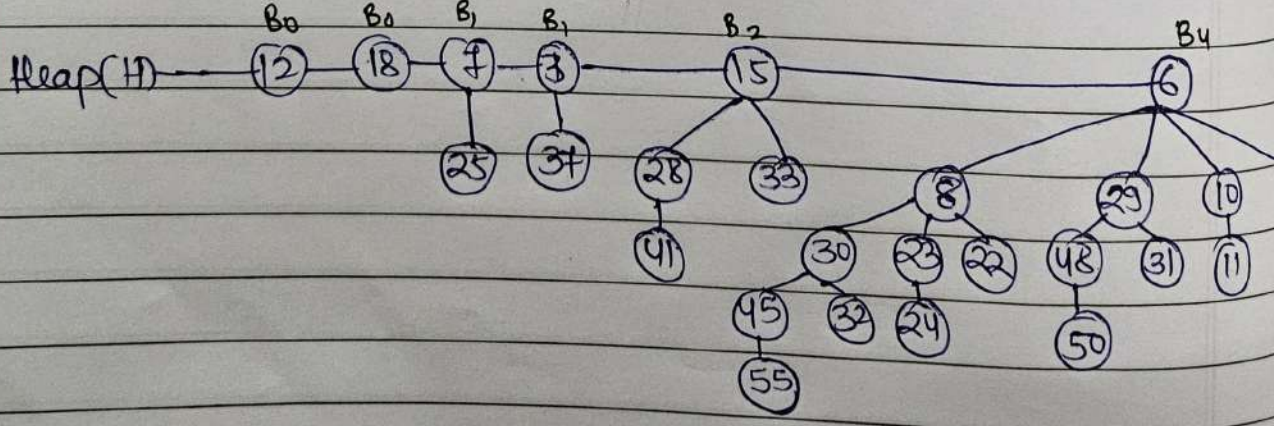


=> Heap (H_2)



③ union of two Binomial Heap :-

Heap (H)



(4) Inserting a Node in Binomial - Heap

=> Heap(H) — 12 (B0) — 7 (B1) — 15 (B2); 25, 28, 33; 41

I want to insert Heap(H2) 17 B0

=> Heap(H) — 17 (B0) — 12 (B0) — 7 (B1) — 15 (B2); 25, 28, 33; 41

∴ (2 - B0 are at same level). (Apply rule)

=> Heap(H) — 12 (B1) — 7 (B1) — 15 (B2); 17, 25, 28, 33; 41

∴ (2 - B1 comes at same level) (Apply rule)

=> Heap(H) — 7 (B2) — 15 (B2); 12, 25, 28, 33; 17, 41

∴ (2 - B2 comes at same level) (Apply rule)

=> Heap(H) — 7 (B3); 15, 12, 29; 28, 33, 17; 41

∴ (Final tree).

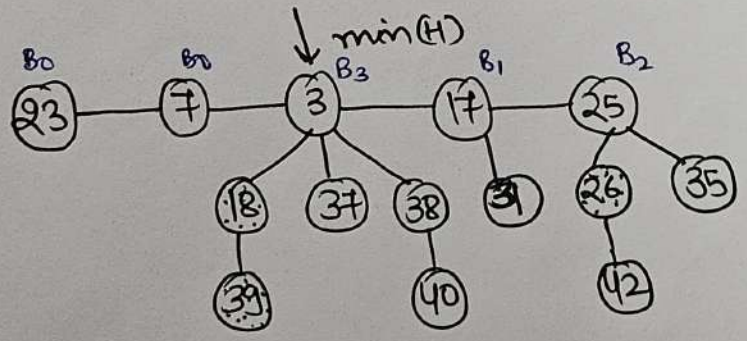
⊙ Fibonacci Heap:-

⊙ Is a collection of "min-heap" ordered Tree.
→ Not constrained to be Binomial Tree.

⊙ properties

1. More than one tree of same order (degree).
2. F-heap is Rooted But unordered.
3. Roots & Siblings are "Circular Doubly linked".
4. Each Node having Degree = No. of children / Descendent.
5. Each Node Marked. or unmarked (true false).
   Newly Node would be unmarked.
6. F-heap (H) :→
   - 6A) min(H) → point to min Rooted node.
   - 6B) n(H) → Total no. of node.
   - 6C) t(H) → Total no. of Rooted Node.
   - 6D) m(H) → Total no. of Marked Node.
   - 6E) ϕ(H) → potential function → ϕ(H) = t(H) + 2m(H).

Example:-



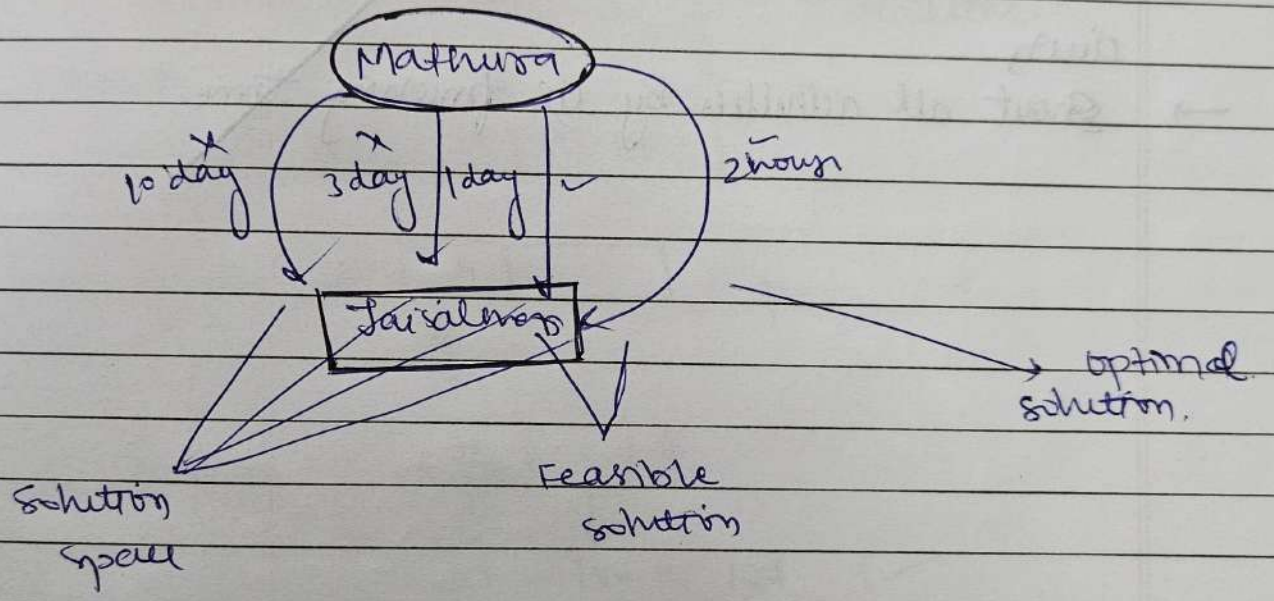
⇒ min(H) → 3.
⇒ n(H) → 14.
⇒ t(H) → 5
⇒ m(H) → 3
⇒ ϕ(H) → t(H) + 2m(H)
= 5 + 2*3
= 5 + 6
= 11.

Greedy Approch

⊙ Greedy Approch :

⊙ The Greedy Approach is a problem - solving method where, at each step, you choose the best possible option, hoping that it will lead to the global optimum (best overall solution).

→ Feasible solution → Solution that satisfied the conditions.
→ optimal solution → Maximization ⁄ minimization.
→ Solution space. → All solution / superset of solution.



1. Activity solution problem.
2. Job sequencing with deadline.
3. Fractional knapsack problem.
4. Huffman coding.
5. Spanning tree / minimum cost.
6. Single source shortest path.

Job Sequence with Deadline

classmate
Date
Page

① Job sequencing problem:-

→ With Deadline.

⊙ In this algorithm each job has a deadline and a profit associated with it.

→ Goal:-
- Each job takes one unit of time.
- No two jobs overlap.
- Total profit is maximized, ensuring that each job is completed before its deadline.

① Ex:-

Job →	J_1	J_2	J_3	J_4
Deadline →	2	1	2	1
profit →	100	10	15	27

person 1. [J_1 |] ✗

↓

100

person 2. [J_4 | J_1]

↓ ↓

27 + 100 = 127. ✓

② Ex:-

Job →	J_1	J_2	J_3	J_4	J_5	J_6
profit →	24	18	22	30	12	10
Deadline →	5	3	3	2	4	3

⇒

person 1 [J_4 | J_3 | J_1 | J_5]

↓ ↓ ↓ ↓

30 + 22 + 24 + 12 = 88 ✗

person 2 [J_4 | J_3 | J_2 | J_5 | J_1]

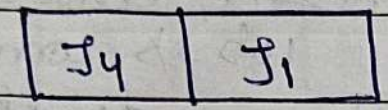
↓ ↓ ↓ ↓ ↓

30 + 22 + 18 + 12 + 24 = 106 ✓

③ Ex

	J_1	J_2	J_3	J_4
profit	50	15	10	25
deadline	2	1	2	1

1. uniprocessor
2. No preemption.
3. Every Job take 1 unit of time.



$25 + 50 = 75$

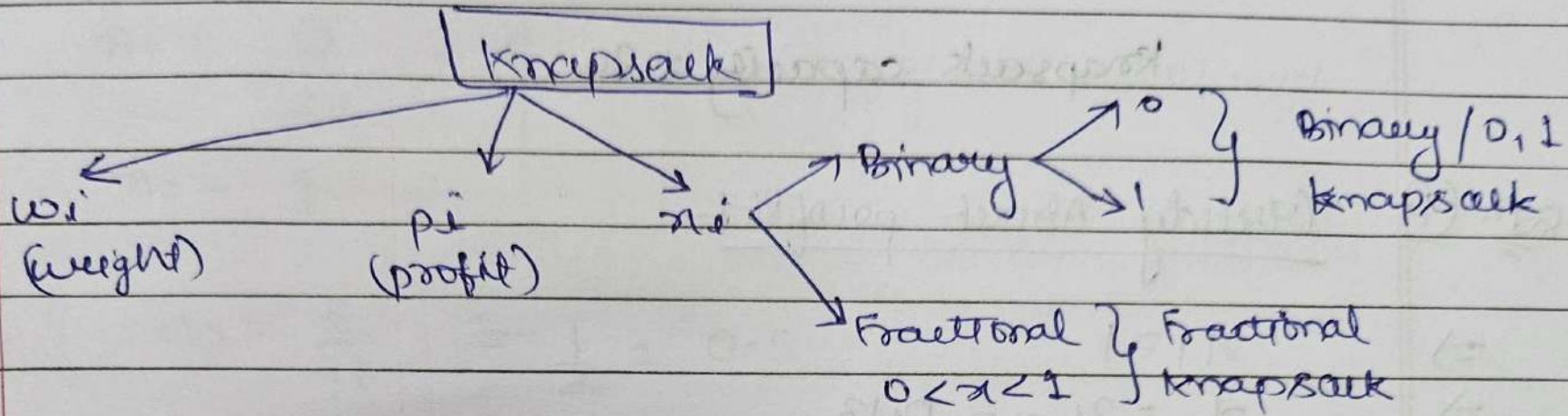
→ $n \log n$

→ $O(n^2)$

Fractional knapsack

⊙ Knapsack problem?

⊙



⊙ Fractional knapsack?

→ O(n log n).

Qw1: Ex1 problem 1:- find the max. profit

	O_1	O_2	O_3
weight →	18	15	10
profit →	25	24	15

knapsack capacity = 20

Soln. (1) Greedy About profit

$\Rightarrow x_1 = 1$

$\Rightarrow x_2 = 2/15 = 0.13$

$\Rightarrow x_3 = 0/10 = 0$

$\Rightarrow$ profit $= \sum_{i=0}^{n} P_i x_i = P_1x_1 + P_2x_2 + P_3x_3$

$\Rightarrow = 25 \times 1 + 24 \times \frac{2}{15} + 15 \times 0$

$\Rightarrow = 25 + \frac{48}{15} + 0$

$\Rightarrow = 25 + 3.12 + 0$

$\Rightarrow = \boxed{28.12}$

Soln. (2) Greedy About weight (put max no. of object in knapsack)

$\Rightarrow x_1 = 0$

$\Rightarrow x_2 = 10/15 = 0.67$

$\Rightarrow x_3 = 1$

$\Rightarrow$ profit $= 0 \times 25 + \frac{10}{15} \times 24 + 1 \times 15$

$\Rightarrow = 0 + \frac{240}{15} + 15$

$\Rightarrow = \boxed{31.08}$

classmate
Date
Page

Sol (3) Greedy About profit | weight :

⇒ $P_1/w_1 = 25/18 = 1.38$

⇒ $P_2/w_2 = 24/15 = 1.60$

⇒ $P_3/w_3 = 15/10 = 1.50$

⇒ $x_1 = \frac{0}{25} = 0$

⇒ $x_2 = 1$

⇒ $x_3 = \frac{5}{10} = \frac{1}{2} = 0.5$

⇒ profit = $0 \times 25 + 1 \times 24 + 0.5 \times 15$

= $0 + 24 + 0.5 \times 15$

= $\boxed{31.5}$

⇒ method 1 = 28.12

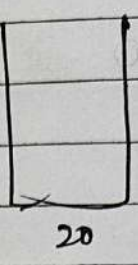
⇒ method 2 = 31.08

⇒ method 3 = 31.50 ⟶ Max.

Qu2. knapsack problem

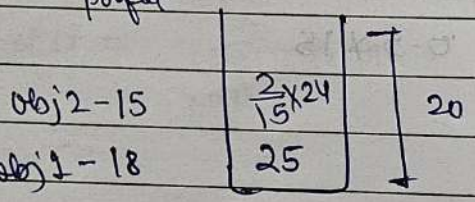
object	1	2	3
profit	25	24	15
weights	18	15	10

knapsack capacity = 20



1. Greedy about profit

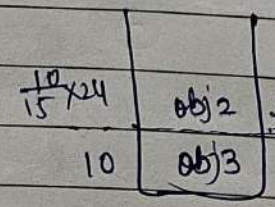
profit



$\Rightarrow 25 + \frac{2}{15} \times 24$

$\Rightarrow 25 + \frac{48}{15} = 28.2$

2 Greedy about weights (ws)



$15 + \frac{10}{15} \times 24 = 31$

3. profit/ weight

1.	1·3
2.	1·6
3.	1·5

5	obj 3	$\frac{5}{10} \times 15$
15	obj 2	24

$$\frac{5}{10} \times 15 + 24 = 7.5 + 24$$
$$= 31.5$$

Minimum Spanning Tree

Date
Page

⊙ Minimum Spanning Tree:-

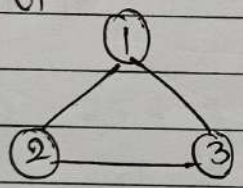
⊙ It is a tree that connects all the nodes (vertices) of a weighted graph using the smallest total weight of edges and no cycle.

$E = V - 1$ ← no. of vertices
↳ edges

→ A connected subgraph H of given graph G is said to be spanning tree iff.

1. H should contains all vertices of G.
2. H should contain $(n-1)$ edges.
3. H should be connected.

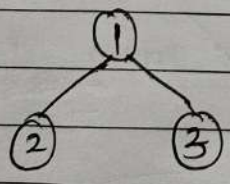
= G

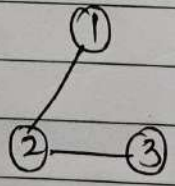


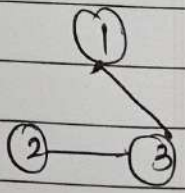
$n = 3$
$e = 3$

=)

= H H H



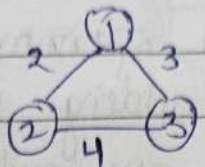




Note :- Complete tree K = n^{n-2} spanning tree.

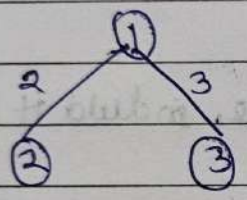
Note :- Edges remove = $e - V + 1$
↳ edges → vertices

minimum cost spanning tree

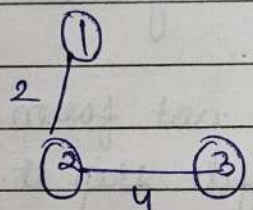
Ex: We have K_3, 

→ weighted graph

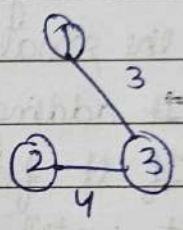
Spanning tree = n^{n-2}
= 3^{3-2}
= 3

H_1 

5

H_2 

6

H_3 

7

min = H_2

⇒ But when a graph has many vertices and edges, it's not easy to manually choose which edges form the minimum spanning tree.

So, we use algorithm to find it step by step in a smart and systematic way,

1. Prim's Algorithm.
2. Kruskal's Algorithm.

Kruskal's Algorithm

Kruskal's Algorithm

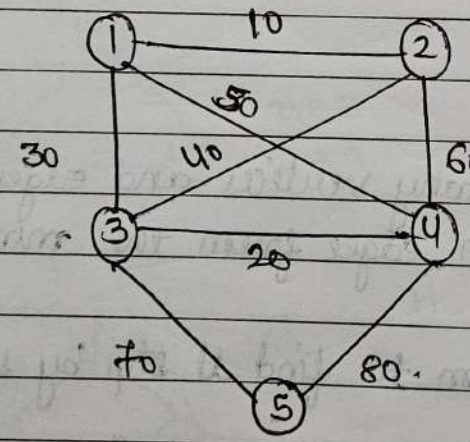
- Use to find minimum spanning tree.

→ This algo works by adding the smallest edges first, one by one without forming any cycle, until all vertices are connected.

Steps:

1. List all edges.
2. Sort all edges in increasing order.
3. Pick the smallest edge.
   - If adding it does not form a cycle, include it in MST.
   - If it forms a cycle skip it.
4. Repeat until MST has V-1 edges

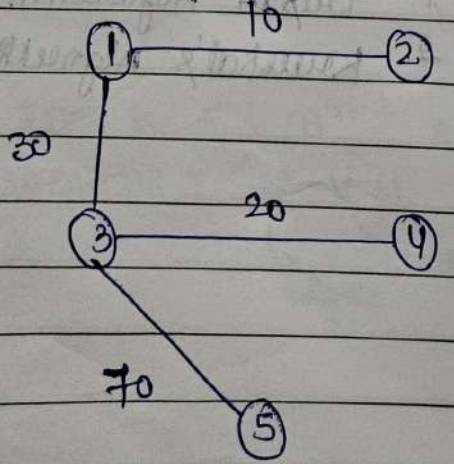
Example:



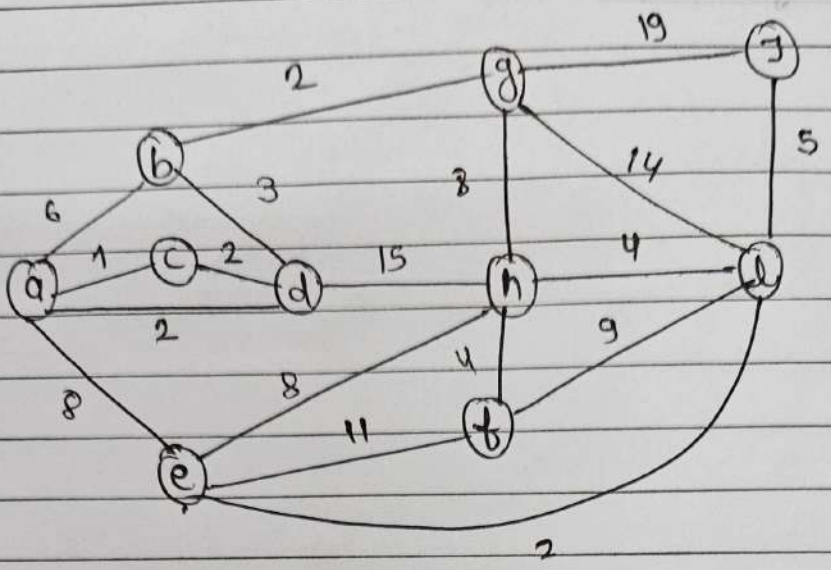
$E = V-1$

$E = 5-1$

$E = 4$

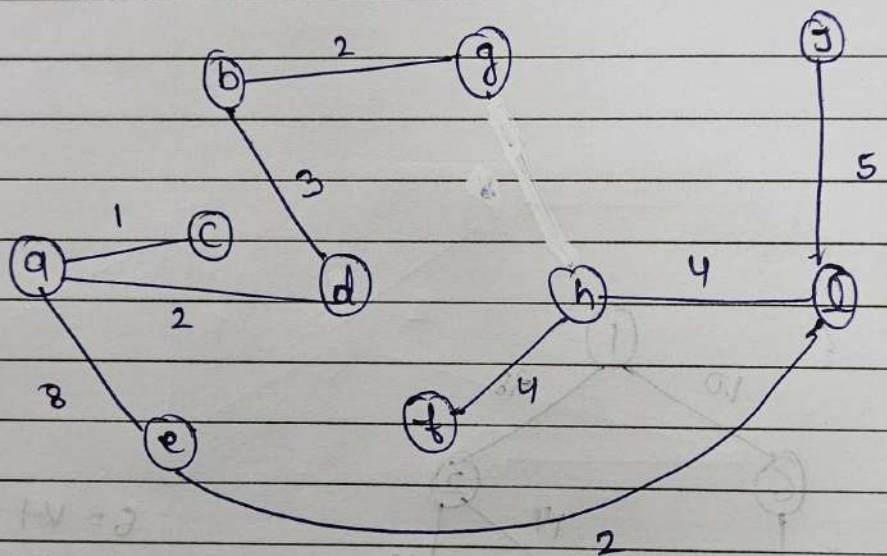


Ex2



E=10-1

E=9



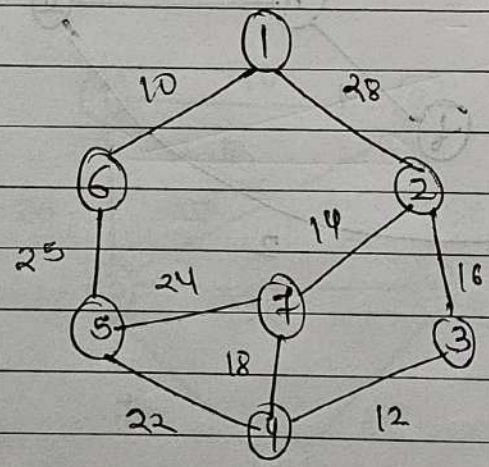
→ Time complexity:

O(nloge)

Prim's Algorithm

① Prim's Algorithm

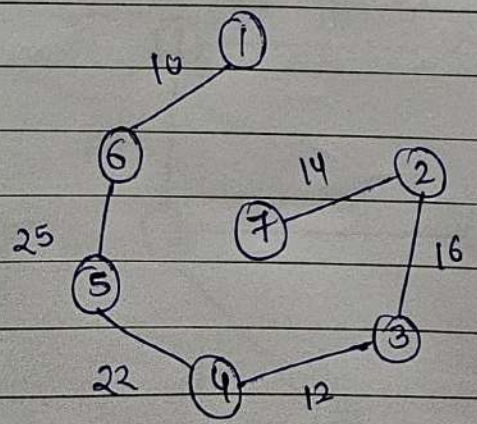
Ex:-



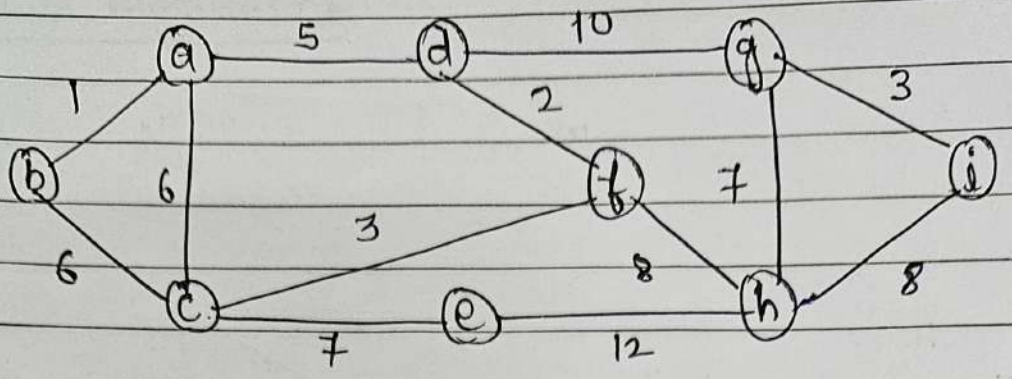
$E = V-1$

$E = 7-1$

$E = 6$



Ex2.

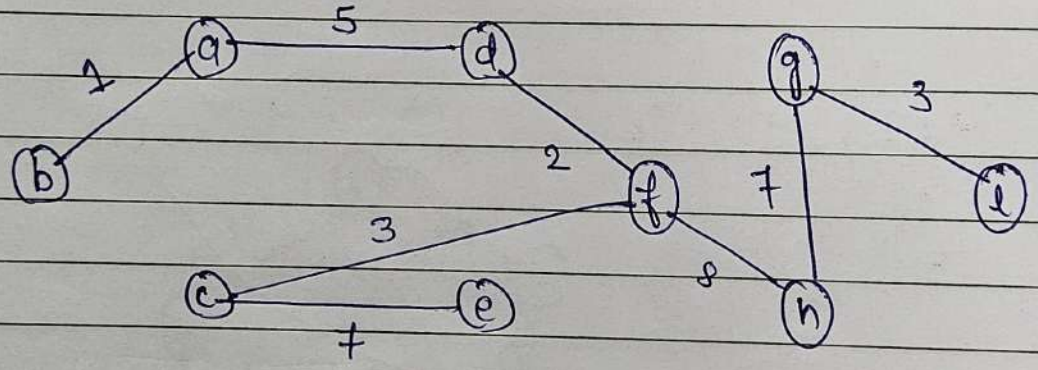


⇒ vertices = 9

⇒ edges = 12

⇒ edges in min. spa. tree = (V-1) = 9-1

= 8



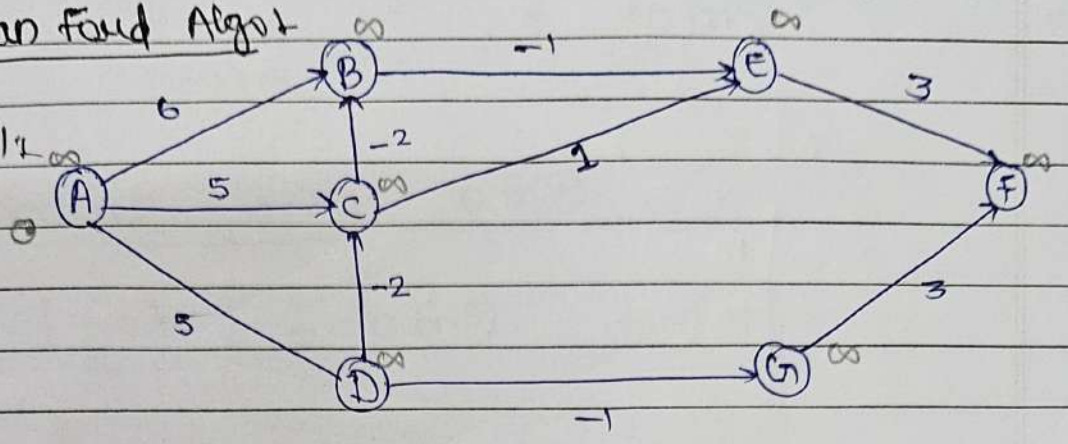
Single source shortest paths

classmate
Date
Page

⊙ Single Source Shortest path :-

Bellman Ford Algo :-

⇒ Example 1 :-



1. Comes under Dynamic programming.
2. Total No. of Iteration
   $= |V| - 1$

3. Relaxation
   If $d(u) + c(u,v) < \infty$
   then,
   $d(v) = d(u) + c(u,v)$

4. For all Negative weight : (Except) :- Not with Negative weighted cycle.

⇒ For example 1 :-

⇒ Iteration ⇒ $|V| - 1$ ⇒ $7 - 1 = \boxed{6}$.

= Time complexity :-

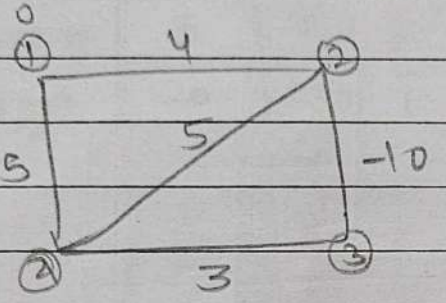
$T = O(V \cdot E)$ ← edges.
↳ vertex

$= O(n^2)$.

= For complete graph :- $O(n^3)$

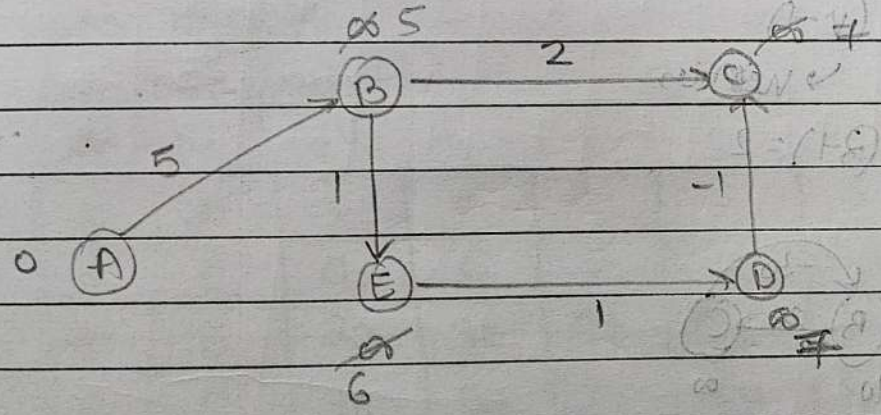
$\frac{E \cdot V(V-1)}{2} = O(n^3)$.

Ex-1.



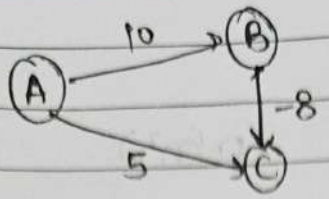
4-1 = 3 Iteration

Ex-2.



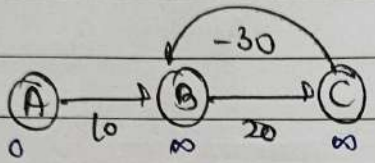
5-1 = 4.

Ex1.



	A	B	C
		∞	∞
A, C		10	5
		10	5

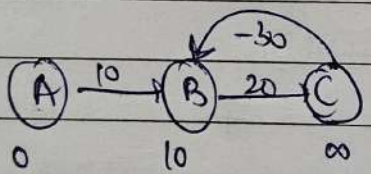
Ex2.

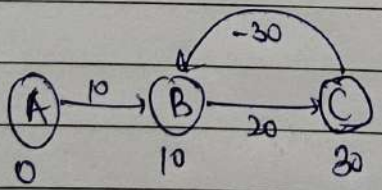


relax (V-1)

↳ vertices

relax = (3-1) = 2

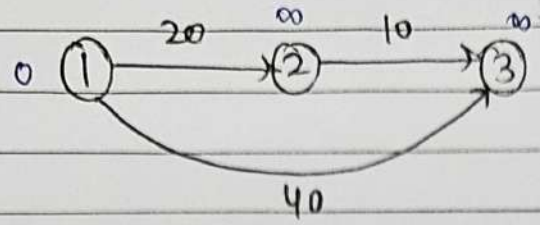




Dijkstra's Algo.

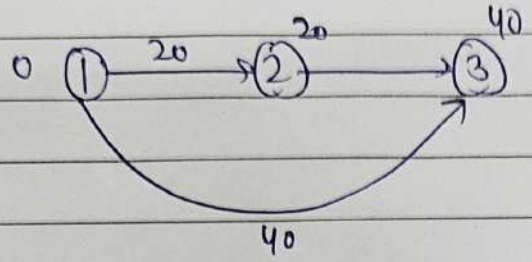
Date / /
Page No.
Shivalal

Ex1.



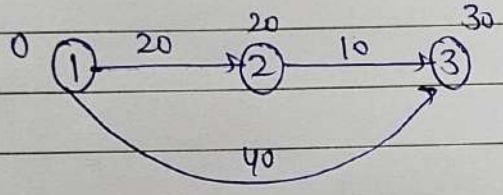
At source = 1.

Step1.

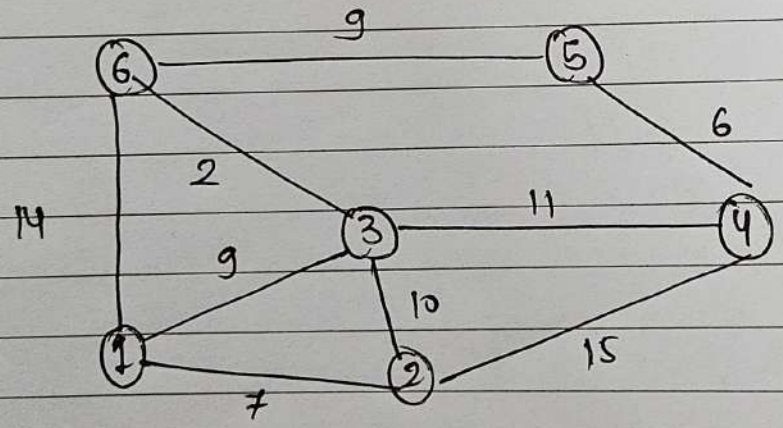


At source = 2

Step2.



Ex2.

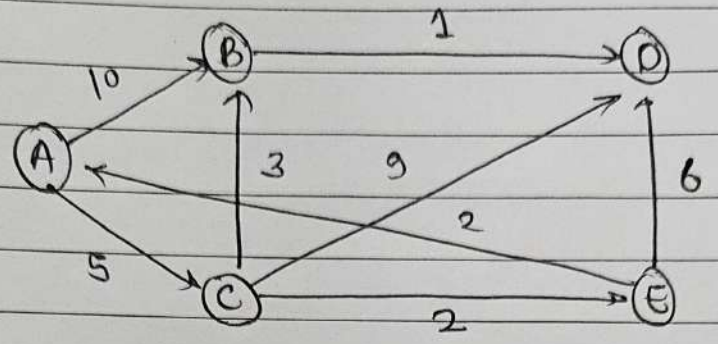


→ Undirected (∴ known)

Source	Destination				
1	2	3	4	5	6
	∞	∞	∞	∞	∞
	(7)	9	∞	∞	14
1,2	(7)	(9)	22	∞	14
1,2,3	(7)	(9)	20	∞	(11)
1,2,3,6	(7)	(9)	(20)	20	(11)

	2	3	4	5	6
1,2,3,4,6	(7)	(9)	(20)	(~~20~~)	(11)
1,2,3,6,4,5	(7)	(9)	(20)	(20)	(11)

Ex3



Source	Destination			
A	B	C	D	E
A	∞	∞	∞	∞
A	10	(5)	∞	∞
A,C	~~10~~	(5)	~~14~~	(7)
A,C,E	(8)	(5)	13	(7)
A,C,E,B	(8)	(5)	(9)	(7)
A,C,E,B,D				

Dynamic programming Applications

0/1 knapsack

classmate
Date
Page

⊙ 0/1 knapsack

⊙

Prob:

obj	1	2	3	4
weights	5	2	4	3
profit	20	30	15	10

Capacity = 7

rows = object +1
columns = capacity +1

	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	20	20	20
2	0	0	30	30	30	30	30	50
3	0	0	30	30	30	30	45	50
4	0	0	30	30	30	40	45	(50)

↑ maximum profit possible

1. max profit = 50.
2. object = obj 1 + obj 2
   set = {1, 1, 0, 0}.

Ques2

object	profit	weight
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

capacity = 10

rows = obj+1

columns = capacity+1

	0	1	2	3	4	5	6	7	8	9	10	11
0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	1	1	1	1	1	1	1	1	1
2	0	1	6	7	7	7	7	7	7	7	7	7
3	0	1	6	7	7	18	19	24	24	24	24	24
4	0	1	6	7	7	18	22	28	28	29	29	40
5	0	1	6	7	7	18	22	28	28	34	35	40

max profit

(5,6)

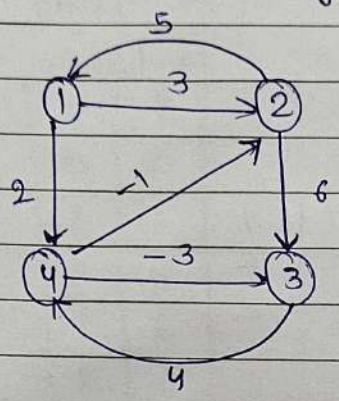
⊙ All pair shortest paths:-

Warshall's and Folyd's algorithms:-

⊙ Floyd-warshall is a dynamic-programming algorithms used to find the shortest distance between every pair of vertices in a weighted graph.

→ Does not work with negative cycles.

Ex:1



$D_0 =$

	1	2	3	4
1	0	3	∞	2
2	5	0	6	∞
3	∞	∞	0	4
4	∞	-1	-3	0

∴ (weight matrix)

$D_1 =$

	1	2	3	4
1	0	3	∞	2
2	5	0	6	7
3	∞	∞	0	4
4	∞	-1	-3	0

∴ (via -1)

$D_2 =$

	1	2	3	4
1	0	3	9	2
2	5	0	6	7
3	∞	∞	0	4
4	4	-1	-3	0

∴ (via 1,2)

$D_3 =$

	1	2	3	4
1	0	3	9	2
2	5	0	6	7
3	∞	∞	0	4
4	4	-1	-3	0

$\therefore (w_{ij} = 1, 2, 3)$

$D_4 =$

	1	2	3	4
1	0	+1	-1	2
2	5	0	4	7
3	8	3	0	4
4	4	-1	-3	0

$(\therefore w_{ij} = 1, 2, 3, 4)$

⊙ Longest Common Subsequence:-

⊙ It is a classic dynamic programming problem used to find the longest sequence that appears in both strings in the same order but not necessarily contiguously.

Ex:- DNA

Ex ⇒

Let X: A B C D H F
Y: A E D F K

⇒ LCS = ADF.

X: A G G T A B
Y: G X T Y A Y B

LCS = G T A B

① Longest common subsequence

Ques

$X \rightarrow \{ B, A, C, D, B \}$

$Y \rightarrow \{ B, D, C, B \}$

X ↓ \ y →		B	D	C	B
	B	0	0	0	0
	A	0			
	C	0			
	D	0			
	B	0			

(X)

Y → \ X ↓		0	B	D	C	B
		0	0	0	0	0
	B	0	↖1	←1	←1	↖1
	A	0	↑1	↑1	↑1	↑1
	C	0	↑1	↑1	↖2	←2
	D	0	↑1	↖2	↑2	↑2
	B	0	↖1	↑2	↑2	↖3

(✓)

⊙ Matrix chain multiplication :-

$$A = \begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \end{bmatrix}_{2\times3} \qquad B = \begin{bmatrix} b_1 & b_2 \\ b_3 & b_4 \\ b_5 & b_6 \end{bmatrix}_{3\times2}$$

$$A \cdot B = \begin{bmatrix} c & d \\ f & g \end{bmatrix} \; e \Big\}$$

$= A_{p\times q} \qquad B_{q\times r}$

$= (A \cdot B)_{p\times r}$

= Scalar multiplication = (pqr)

$= 2\times3\times2 = 12$

Ex:

$A_{2\times5} \quad B_{5\times10} \quad C_{10\times3}$

(180) (160)

A(BC) (AB)C

$A_{2\times5} \quad (BC)_{5\times3}$

$\downarrow$

$5\times10\times3 = 150$

$A(BC)_{2\times3}$

$\downarrow$

$2\times5\times3 = 30$

$= 150 + 30 = 180$

$(AB)_{2\times10}$

$\downarrow$

$2\times5\times10 = 100$

$(AB)C_{2\times3}$

$\downarrow$

$2\times10\times3 = 60$

$= 160$

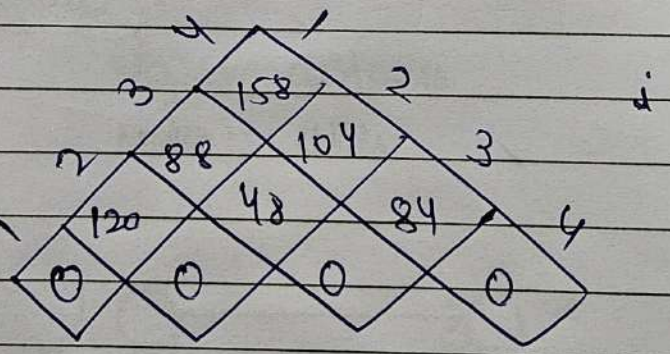
Ques #5 MCM - matrix chain multiplication

Matrix	Dimension
A_1	5×4
A_2	4×6
A_3	6×2
A_4	2×7

$P_0 = 5$
$P_1 = 4$
$P_2 = 6$
$P_3 = 2$
$P_4 = 7$

→ There are so many combinations
It is difficult to manually compair
So, we use formula

$$m[i,j] = \min\{ m[i,k] + m[k+1,j] + p_{i-1} p_k p_j \}$$
where $i \le k \le j-1$



$m[1,2]$ = By applying formula.
$i=1$
$j=2$
$1 \le k \le 1 = 1$

$m[1,2] = \min\{ m[1,1] + m[2,2] + P_0 P_1 P_2 \}$

Ques2

$= \min\{0+0+5(4)(6)\}$

$= 120$

⇒ So,

$m[2,3] = 48$

$m[3,4] = 84$

⊙ Backtracking, Branch and Bound Applications

Branch and Bound:

⊙ Travelling salesman problem:

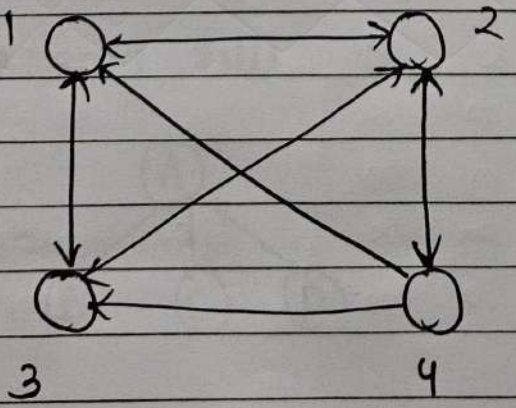
⊙ This problem asks:
A salesman must visit every city exactly once and return to the starting city while minimizing the total travel cost/distance.

→ In short:
- visit all cities
- No city visited twice
- Come back to start
- Minimum total distance

→ Representation:

Graph:
edges: distance
Nodes: cities

Ex:



	1	2	3	4
1	0	10	15	20
2	5	0	25	10
3	15	30	0	5
4	15	10	20	0

Ex 2:

	A	B	C	D	
A	∞	10	5	3	3
B	8	∞	9	7	7
C	1	6	∞	9	1
D	2	3	8	∞	2
	+	3	5	3	16

Step 1:

Row reduction matrix

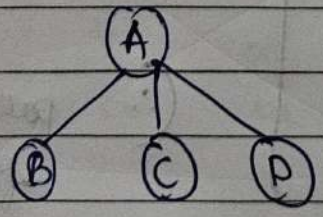
	A	B	C	D	
A	∞	7	2	0	
B	1	∞	2	0	
C	0	5	∞	8	
D	0	1	6	∞	
	0	1	2	0	3

then

Step 2 column reduction matrix

	A	B	C	D	
A	∞	6	0	0	
B	1	∞	0	0	∴ (parent matrix for path 1.
C	0	4	∞	8	
D	0	0	6	∞	

State space tree



find A–B | A–C | A–D

(i) A — B

	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	0	0
C	0	∞	∞	8
D	0	∞	4	∞

reduction in (A-B)

Cost = 16 + 0 + 6

Total reduction in parent

distance (A-B)

Cost = 22

(ii) A — C

	A	B	C	D
A	∞	∞	∞	∞
B	1	∞	∞	0
C	∞	4	∞	8
D	0	0	∞	∞

4

	A	B	C	D
A	∞	∞	∞	∞
=B	1	∞	∞	0
C	∞	0	∞	4
D	0	0	∞	∞

Cost = 16 + 4 + 0

Cost = 20

(iii) A — D

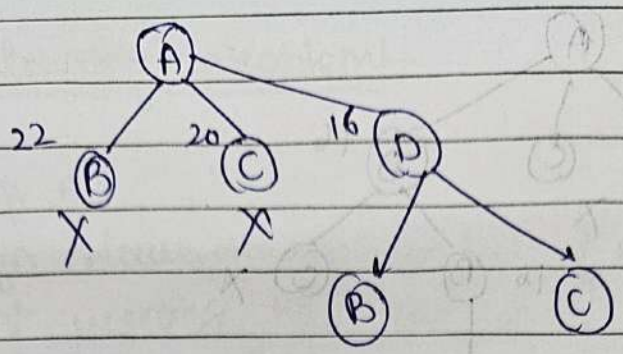
	A	B	C	D
A	∞	∞	∞	∞
B	1	∞	0	∞
C	0	4	∞	∞
D	∞	0	4	∞

(∵ parent matrix for path 2)

cost = 16 + 0 + 0

= 16.

According to this



To find

A – D – B | A – D – C

(i) A – D – B

	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	0	∞
C	0	∞	∞	∞
D	∞	∞	∞	∞

Cost = 16 + 0 + 0

= 16

(ii) A – D – C

	A	B	C	D
A	∞	∞	∞	∞
B	1	∞	∞	∞
C	∞	4	∞	∞
D	∞	∞	∞	∞

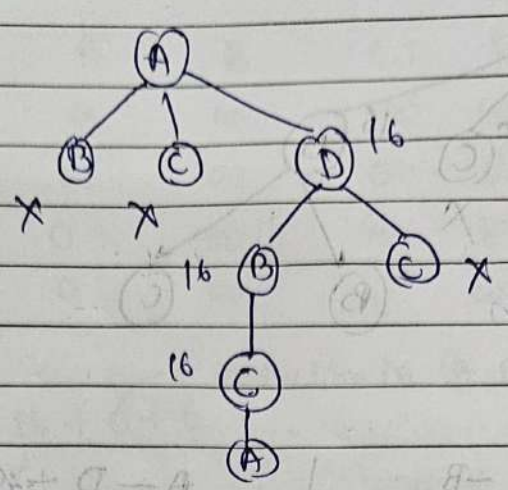
S

	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	∞	∞
C	∞	0	∞	∞
D	∞	∞	∞	∞

Cost = 16 + 5 + 4

= 25

Accoseding to this



(i) find

A - D - B - C

⇒ cost = 16

(ii) then find

A - D - B - C - A

cost = 3+3+9+1 = 16

Back tracking

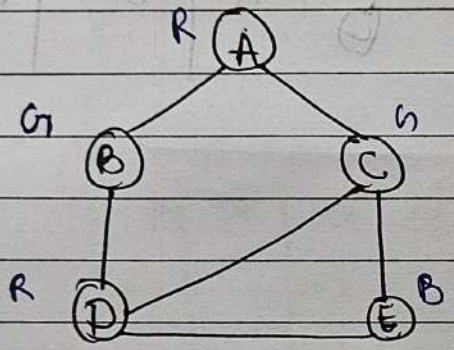
⊙ Graph colouring problem

⊙ This problem is:

Assign colours to the vertices of a graph such that no two adjacent vertices have the same colour, using the minimum numbers of colours.

→ The minimum numbers of colours is called the chromatic Numbers.

Ex: (i)



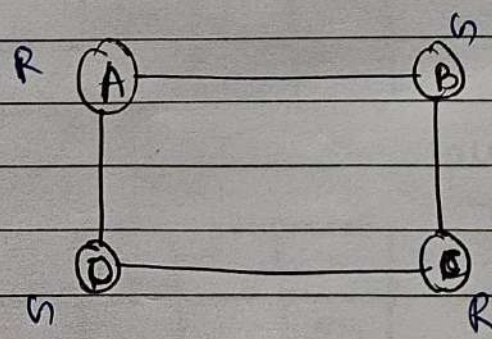
color = { Red, Green, Blue }
(R, G, B)

A	B	C	D	E
R	G	G	R	B
G	R	R	G	B

(3^m → Brute force App.)

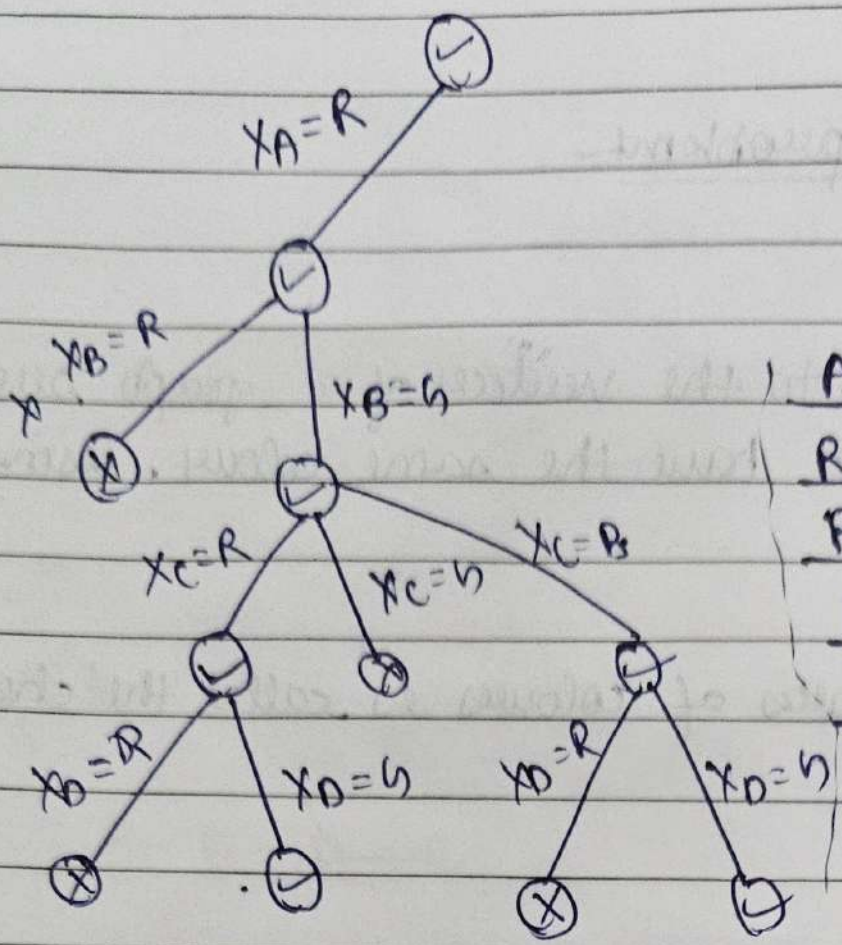
multiple answers.

(ii)



↓

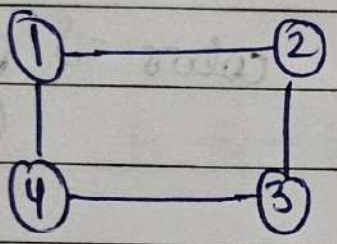
C.N = 2 → (colours)



A	B	C	D
R	G	R	G
R	G	R	B
R	G	B	G
R	G	B	G
R	B	R	G
R	B	R	B

$\{0, 00\}$.

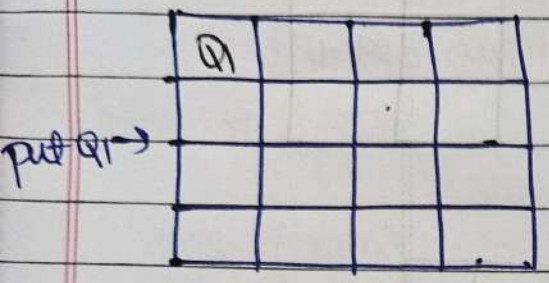
(2)



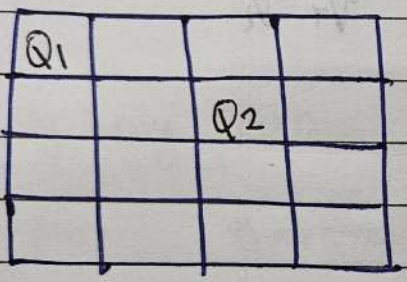
⊙ n- Queen problem

⇒ 4 - Queen problem

→ No Queen attack means
1. no 2 Queen in one row
2. no 2 Queen in one column
3. no 2 Queen in one diagonal.

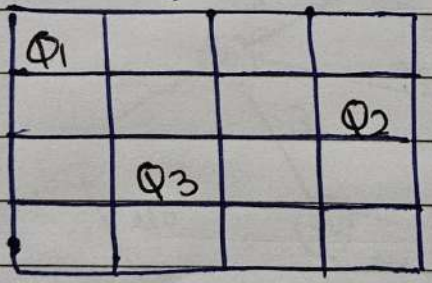


put Q_1



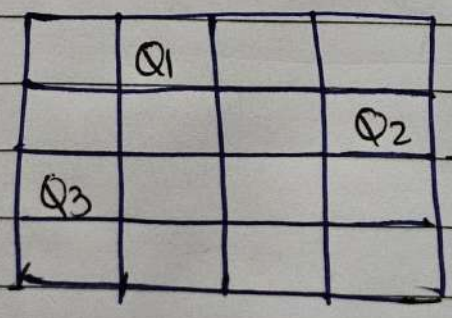
put Q_2

not possible to put Q_3 then back track

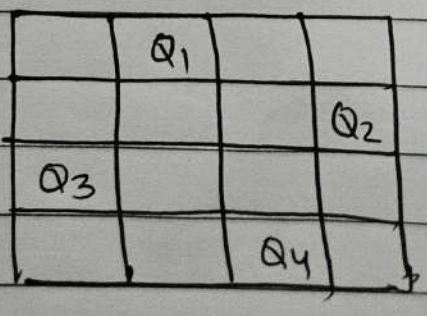


put Q_3

problem to put Q_4 then backtrack.

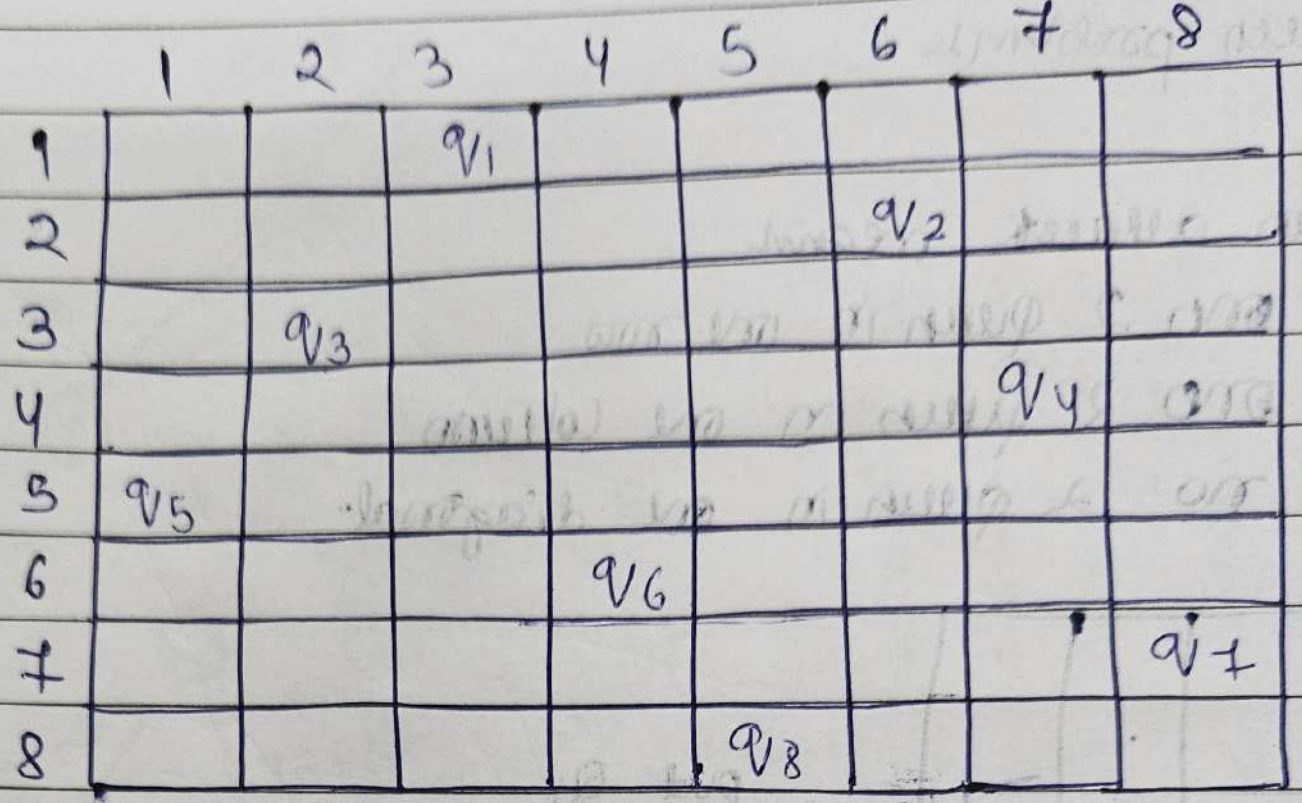


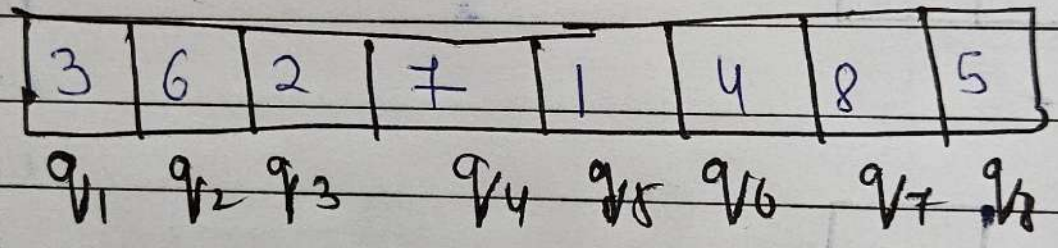
put Q_4 finally



(2, 4, 1, 3)

8 - queen problem





① Sum of Subsets:

②

→ Answer in True or False.

Ex:- $S = \{10, 20, 30, 40\}$

sum $m = 50$

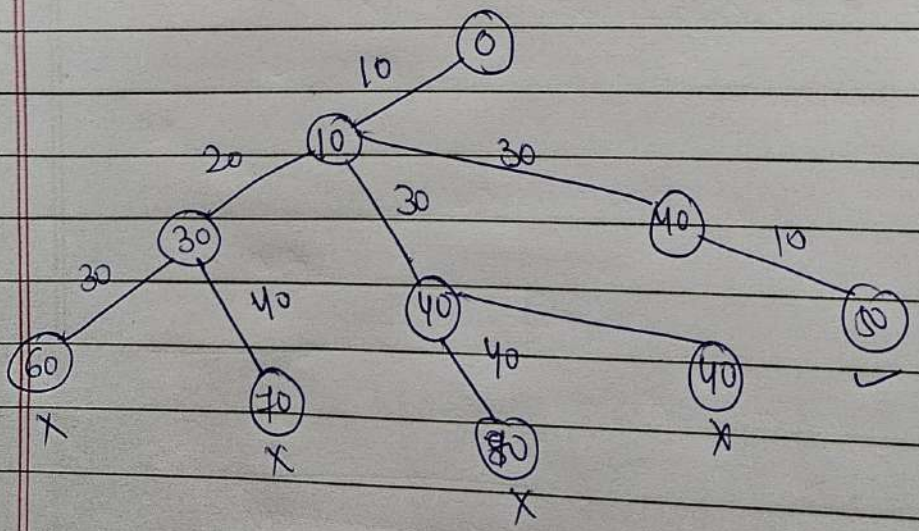
SOS (n, m)

$\{20, 30\}$

$\{10, 40\}$

⇒ By using Backtracking.

State space tree



State Space tree